🎯 LAB 4: REST API DESIGN & CRUD MASTERY

Weeks 6-7 | Build Professional REST APIs with Proper Design

Instructor: Mohamed Amine Marzouk
Email: med.amine.mzk@horizon-tech.tn
Difficulty: ⭐⭐ Intermediate
Time Estimate: 10-12 hours over 2 weeks
Learning Outcome: AA3 - REST APIs & CRUD Operations
Date: January 25, 2026

📍 WHERE WE ARE IN THE COURSE

You've completed **LAB 1, LAB 2, & LAB 3**! You now have:

- ☑ Deep async/await knowledge
- ☑ Modular code organization
- ☑ Express.js web servers
- ☑ Routing & middleware
- ☑ MVC architecture

Now it's time to design PROFESSIONAL REST APIs following industry standards!

🎬 WHY THIS MATTERS - THE REAL PROBLEM

The Problem: Bad API Design

Imagine an API without standards:

```
❌ BAD API Design:

GET  /getalluser           ← Inconsistent naming
POST /user/insert          ← Wrong verbs
GET  /get_users_list       ← Snake case
DELETE /remove-user/1      ← Inconsistent
GET  /user?id=1&id=2&id=3  ← Confusing
POST /user                 ← No status code
PUT  /user/1               ← Returns nothing
GET  /error                ← Unclear endpoints

Problems:
❌ Hard to understand
❌ Inconsistent conventions
❌ Wrong HTTP methods
❌ Unclear status codes
❌ Bad error messages
```

```
❌ No documentation
❌ Breaks on edge cases
```

The Solution: REST API Standards

```
✅ GOOD REST API Design:

GET    /api/users           ← List all
GET    /api/users/:id       ← Get one
POST   /api/users           ← Create
PUT    /api/users/:id       ← Update
DELETE /api/users/:id       ← Delete

Standards:
✅ Resource-oriented
✅ Consistent naming
✅ Correct HTTP methods
✅ Proper status codes
✅ Clear error messages
✅ Documented (Swagger)
✅ Handles edge cases
✅ Industry standard
```

🌍 REAL-WORLD COMPANIES USING REST

Company	API	Why REST Matters
GitHub	Public REST API	Developers build tools on top
Stripe	Payment API	$50B payments processed
Slack	Workspace API	Integrations power ecosystem
Twitter	Social API	Millions of requests/second
AWS	Cloud APIs	Manages infrastructure

🚀 WHAT YOU'LL BUILD

By the end, you'll create:

1. ✅ Professional REST API (Events)
2. ✅ All CRUD operations properly designed
3. ✅ Correct HTTP status codes
4. ✅ Comprehensive error handling
5. ✅ Request validation
6. ✅ API documentation (Swagger/OpenAPI)
7. ✅ Professional response formats

API Endpoints:

```
GET    /api/v1/events              → 200 OK
GET    /api/v1/events/:id          → 200 OK or 404
GET    /api/v1/events?status=upcoming → 200 OK (filtered)
POST   /api/v1/events              → 201 Created
PUT    /api/v1/events/:id          → 200 OK
DELETE /api/v1/events/:id          → 204 No Content
```

☑ PRE-LAB CHECKLIST

Before starting:

- ☐ Completed LAB 1, 2, 3
- ☐ Understand HTTP methods (GET, POST, PUT, DELETE)
- ☐ Understand status codes (2xx, 4xx, 5xx)
- ☐ Node.js & Express working
- ☐ npm installed
- ☐ Code editor ready
- ☐ 10-12 hours available
- ☐ Ready to build professional APIs!

📚 REST API PRINCIPLES

1. Resource-Oriented Design

Everything is a **resource**:

```
Resources:
/api/events        ← Resource: events (collection)
/api/events/1      ← Resource: specific event
/api/users         ← Resource: users
/api/users/5       ← Resource: specific user
```

2. HTTP Methods (Verbs)

Methods tell what to DO with resources:

```
GET     → Read (safe, idempotent)
POST    → Create (not idempotent)
PUT     → Update (idempotent)
DELETE  → Delete (idempotent)
PATCH   → Partial update
```

3. Status Codes

Numbers tell the RESULT:

```
2xx Success:
200 OK              ← Request successful
201 Created         ← Resource created
204 No Content      ← Success, no response body

4xx Client Error:
400 Bad Request     ← Invalid data
401 Unauthorized    ← Not authenticated
403 Forbidden       ← Not authorized
404 Not Found       ← Resource doesn't exist
409 Conflict        ← Resource conflict

5xx Server Error:
500 Internal Error  ← Server problem
503 Unavailable     ← Server down
```

4. Response Format

Consistent structure:

```
{
  "success": true,
  "data": { ... },
  "message": "Success message",
  "timestamp": "2026-01-25T16:00:00Z"
}
```

🎯 TASK 1: REST PRINCIPLES & DESIGN

Step 1: Setup Lab 4

DO THIS:

```
# Create Lab 4
cd ~/node-course
cp -r lab-3 lab-4
cd lab-4

# Install express if needed
npm install
```

Step 2: Create REST Constants

Create file: `src/constants/rest.js`

Copy this:

```
// REST API CONSTANTS

export const HTTP_METHODS = {
  GET: "GET",
  POST: "POST",
  PUT: "PUT",
  DELETE: "DELETE",
  PATCH: "PATCH"
};

export const STATUS_CODES = {
  // Success
  OK: 200,
  CREATED: 201,
  ACCEPTED: 202,
  NO_CONTENT: 204,

  // Client errors
  BAD_REQUEST: 400,
  UNAUTHORIZED: 401,
  FORBIDDEN: 403,
  NOT_FOUND: 404,
  CONFLICT: 409,
  UNPROCESSABLE: 422,

  // Server errors
  INTERNAL_ERROR: 500,
  NOT_IMPLEMENTED: 501,
  SERVICE_UNAVAILABLE: 503
};

export const HTTP_MESSAGES = {
  200: "Request successful",
  201: "Resource created successfully",
  204: "Request successful (no content)",
  400: "Invalid request data",
  401: "Authentication required",
  403: "Access denied",
  404: "Resource not found",
  409: "Resource conflict",
  500: "Internal server error",
  503: "Service unavailable"
};

export const API_VERSION = "v1";
```

```
export const API_BASE = `/api/${API_VERSION}`;

export const VALIDATION_RULES = {
  event: {
    title: {
      required: true,
      minLength: 3,
      maxLength: 100,
      type: "string"
    },
    date: {
      required: true,
      type: "string",
      format: "ISO8601"
    },
    location: {
      required: true,
      minLength: 2,
      maxLength: 100,
      type: "string"
    },
    capacity: {
      required: true,
      type: "number",
      min: 1,
      max: 10000
    },
    description: {
      required: false,
      maxLength: 1000,
      type: "string"
    }
  }
};
```

Step 3: Create Response Wrapper

Create file: `src/utils/apiResponse.js`

Copy this:

```
// API RESPONSE WRAPPER
// Standardize all responses

export class ApiResponse {
  constructor(success, data = null, message = "", statusCode = 200) {
    this.success = success;
    this.data = data;
    this.message = message;
    this.statusCode = statusCode;
```

```
    this.timestamp = new Date().toISOString();
  }

  // Success response
  static success(data, message = "Success", statusCode = 200) {
    return new ApiResponse(true, data, message, statusCode);
  }

  // Created response
  static created(data, message = "Resource created successfully") {
    return new ApiResponse(true, data, message, 201);
  }

  // Error response
  static error(message, statusCode = 400, data = null) {
    return new ApiResponse(false, data, message, statusCode);
  }

  // Not found response
  static notFound(resource = "Resource") {
    return new ApiResponse(false, null, `${resource} not found`, 404);
  }

  // Validation error
  static validationError(errors) {
    return new ApiResponse(
      false,
      errors,
      "Validation failed",
      422
    );
  }

  // Paginated response
  static paginated(items, page, limit, total, message = "Success") {
    return {
      success: true,
      data: items,
      pagination: {
        page: parseInt(page),
        limit: parseInt(limit),
        total: total,
        pages: Math.ceil(total / limit),
        hasMore: page * limit < total
      },
      message,
      timestamp: new Date().toISOString()
    };
  }

  // Convert to JSON
  toJSON() {
    return {
      success: this.success,
```

```
      data: this.data,
      message: this.message,
      timestamp: this.timestamp
    };
  }
}

export default ApiResponse;
```

Step 4: Create Validation Service

Create file: `src/services/validationService.js`

Copy this:

```
// VALIDATION SERVICE
// Validate event data

export class ValidationService {
  // Validate event creation/update
  static validateEvent(data, isUpdate = false) {
    const errors = {};

    // Title
    if (!isUpdate || data.title !== undefined) {
      if (!data.title) {
        errors.title = "Title is required";
      } else if (data.title.length < 3) {
        errors.title = "Title must be at least 3 characters";
      } else if (data.title.length > 100) {
        errors.title = "Title must not exceed 100 characters";
      }
    }

    // Date
    if (!isUpdate || data.date !== undefined) {
      if (!data.date) {
        errors.date = "Date is required";
      } else if (isNaN(new Date(data.date).getTime())) {
        errors.date = "Date must be a valid ISO 8601 date";
      } else if (new Date(data.date) < new Date()) {
        errors.date = "Date must be in the future";
      }
    }

    // Location
    if (!isUpdate || data.location !== undefined) {
      if (!data.location) {
        errors.location = "Location is required";
      } else if (data.location.length < 2) {
```

```
        errors.location = "Location must be at least 2 characters";
      } else if (data.location.length > 100) {
        errors.location = "Location must not exceed 100 characters";
      }
    }

    // Capacity
    if (!isUpdate || data.capacity !== undefined) {
      if (data.capacity === undefined) {
        errors.capacity = "Capacity is required";
      } else if (!Number.isInteger(data.capacity)) {
        errors.capacity = "Capacity must be a number";
      } else if (data.capacity < 1) {
        errors.capacity = "Capacity must be at least 1";
      } else if (data.capacity > 10000) {
        errors.capacity = "Capacity must not exceed 10000";
      }
    }

    // Description (optional)
    if (data.description && data.description.length > 1000) {
      errors.description = "Description must not exceed 1000 characters";
    }

    return {
      isValid: Object.keys(errors).length === 0,
      errors
    };
  }

  // Validate ID
  static validateId(id) {
    return !isNaN(parseInt(id)) && parseInt(id) > 0;
  }

  // Validate pagination params
  static validatePagination(page, limit) {
    const errors = {};

    if (!page) {
      page = 1;
    } else if (isNaN(page) || page < 1) {
      errors.page = "Page must be a positive number";
    }

    if (!limit) {
      limit = 10;
    } else if (isNaN(limit) || limit < 1 || limit > 100) {
      errors.limit = "Limit must be between 1 and 100";
    }

    return {
      isValid: Object.keys(errors).length === 0,
      errors,
```

```
      page: parseInt(page),
      limit: parseInt(limit)
    };
  }
}

export default ValidationService;
```

☑ CHECKPOINT 1: REST Principles

- ☐ Understand HTTP methods
- ☐ Know status codes
- ☐ Understand resource-oriented design
- ☐ ApiResponse class created
- ☐ ValidationService created

🎯 TASK 2: IMPLEMENT COMPLETE CRUD

Step 1: Enhanced Event Model

Edit: `src/models/Event.js`

Replace with this:

```
// ENHANCED EVENT MODEL
// With filtering and search

let events = [
  {
    id: 1,
    title: "JavaScript Workshop",
    date: "2026-02-15T10:00:00Z",
    location: "Sfax",
    capacity: 30,
    attendees: 15,
    status: "upcoming",
    description: "Learn modern JavaScript",
    createdAt: "2026-01-01T00:00:00Z"
  },
  {
    id: 2,
    title: "React Conference",
    date: "2026-03-20T14:00:00Z",
    location: "Tunis",
    capacity: 100,
    attendees: 45,
    status: "upcoming",
```

```
      description: "React best practices",
      createdAt: "2026-01-01T00:00:00Z"
    }
  ];

  let nextId = 3;

  export class Event {
    // Get all events with optional filtering
    static getAll(filters = {}, page = 1, limit = 10) {
      let result = [...events];

      // Apply filters
      if (filters.status) {
        result = result.filter(e => e.status === filters.status);
      }

      if (filters.location) {
        result = result.filter(e =>
          e.location.toLowerCase().includes(filters.location.toLowerCase())
        );
      }

      if (filters.search) {
        result = result.filter(e =>
          e.title.toLowerCase().includes(filters.search.toLowerCase())
        );
      }

      if (filters.minCapacity) {
        result = result.filter(e => e.capacity >= parseInt(filters.minCapacity));
      }

      // Sort by date (newest first)
      result.sort((a, b) => new Date(b.date) - new Date(a.date));

      // Paginate
      const total = result.length;
      const start = (page - 1) * limit;
      const paginated = result.slice(start, start + limit);

      return {
        items: paginated,
        total,
        page,
        limit,
        pages: Math.ceil(total / limit)
      };
    }

    // Get by ID
    static getById(id) {
      return events.find(e => e.id === parseInt(id));
    }
```

```
  // Create new
  static create(data) {
    const newEvent = {
      id: nextId++,
      title: data.title,
      date: data.date,
      location: data.location,
      capacity: data.capacity,
      description: data.description || "",
      attendees: 0,
      status: "upcoming",
      createdAt: new Date().toISOString(),
      updatedAt: new Date().toISOString()
    };

    events.push(newEvent);
    return newEvent;
  }

  // Update
  static update(id, data) {
    const event = this.getById(id);
    if (!event) return null;

    Object.assign(event, data);
    event.updatedAt = new Date().toISOString();
    return event;
  }

  // Delete
  static delete(id) {
    const index = events.findIndex(e => e.id === parseInt(id));
    if (index === -1) return null;

    return events.splice(index, 1)[0];
  }

  // Get count
  static getCount() {
    return events.length;
  }

  // Check if exists
  static exists(id) {
    return events.some(e => e.id === parseInt(id));
  }

  // Add attendee
  static addAttendee(id) {
    const event = this.getById(id);
    if (!event || event.attendees >= event.capacity) {
      return null;
    }
```

```
    event.attendees++;
    return event;
  }
}

export default Event;
```

Step 2: Enhanced Controller

Edit: `src/controllers/eventController.js`

Replace with this:

```
// ENHANCED EVENT CONTROLLER
// Professional REST API implementation

import Event from "../models/Event.js";
import ApiResponse from "../utils/apiResponse.js";
import ValidationService from "../services/validationService.js";

export class EventController {
  // GET all events (with filtering & pagination)
  static getAllEvents(req, res) {
    try {
      const { status, location, search, minCapacity, page = 1, limit = 10 } =
req.query;

      // Validate pagination
      const { isValid, errors, page: validPage, limit: validLimit } =
        ValidationService.validatePagination(page, limit);

      if (!isValid) {
        return res.status(400).json(
          ApiResponse.error("Invalid pagination parameters", 400, errors)
        );
      }

      // Get filtered events
      const filters = {};
      if (status) filters.status = status;
      if (location) filters.location = location;
      if (search) filters.search = search;
      if (minCapacity) filters.minCapacity = minCapacity;

      const result = Event.getAll(filters, validPage, validLimit);

      res.status(200).json(
        ApiResponse.paginated(
          result.items,
```

```
            result.page,
            result.limit,
            result.total,
            `Retrieved ${result.items.length} events`
          )
        );
      } catch (error) {
        res.status(500).json(
          ApiResponse.error("Internal server error", 500)
        );
      }
    }

    // GET single event
    static getEventById(req, res) {
      try {
        const { id } = req.params;

        // Validate ID
        if (!ValidationService.validateId(id)) {
          return res.status(400).json(
            ApiResponse.error("Invalid event ID format", 400)
          );
        }

        const event = Event.getById(id);

        if (!event) {
          return res.status(404).json(
            ApiResponse.notFound("Event")
          );
        }

        res.status(200).json(
          ApiResponse.success(event, "Event retrieved successfully")
        );
      } catch (error) {
        res.status(500).json(
          ApiResponse.error("Internal server error", 500)
        );
      }
    }

    // POST create event
    static createEvent(req, res) {
      try {
        const { title, date, location, capacity, description } = req.body;

        // Validate
        const validation = ValidationService.validateEvent({
          title,
          date,
          location,
          capacity,
```

```
        description
      });

      if (!validation.isValid) {
        return res.status(422).json(
          ApiResponse.validationError(validation.errors)
        );
      }

      // Create
      const newEvent = Event.create({
        title,
        date,
        location,
        capacity,
        description
      });

      res.status(201).json(
        ApiResponse.created(newEvent, "Event created successfully")
      );
    } catch (error) {
      res.status(500).json(
        ApiResponse.error("Internal server error", 500)
      );
    }
  }

  // PUT update event
  static updateEvent(req, res) {
    try {
      const { id } = req.params;

      // Validate ID
      if (!ValidationService.validateId(id)) {
        return res.status(400).json(
          ApiResponse.error("Invalid event ID format", 400)
        );
      }

      // Check exists
      if (!Event.exists(id)) {
        return res.status(404).json(
          ApiResponse.notFound("Event")
        );
      }

      // Validate update data
      const validation = ValidationService.validateEvent(req.body, true);

      if (!validation.isValid) {
        return res.status(422).json(
          ApiResponse.validationError(validation.errors)
        );
```

```
      }

      // Update
      const updated = Event.update(id, req.body);

      res.status(200).json(
        ApiResponse.success(updated, "Event updated successfully")
      );
    } catch (error) {
      res.status(500).json(
        ApiResponse.error("Internal server error", 500)
      );
    }
  }

  // DELETE event
  static deleteEvent(req, res) {
    try {
      const { id } = req.params;

      // Validate ID
      if (!ValidationService.validateId(id)) {
        return res.status(400).json(
          ApiResponse.error("Invalid event ID format", 400)
        );
      }

      // Delete
      const deleted = Event.delete(id);

      if (!deleted) {
        return res.status(404).json(
          ApiResponse.notFound("Event")
        );
      }

      res.status(204).send(); // No content
    } catch (error) {
      res.status(500).json(
        ApiResponse.error("Internal server error", 500)
      );
    }
  }

  // GET event statistics
  static getStats(req, res) {
    try {
      const events = Event.getAll({}, 1, 10000).items;

      const stats = {
        totalEvents: Event.getCount(),
        totalCapacity: events.reduce((sum, e) => sum + e.capacity, 0),
        totalAttendees: events.reduce((sum, e) => sum + e.attendees, 0),
        avgCapacity: Math.round(
```

```
            events.reduce((sum, e) => sum + e.capacity, 0) / events.length
          ),
          statusBreakdown: {
            upcoming: events.filter(e => e.status === "upcoming").length,
            ongoing: events.filter(e => e.status === "ongoing").length,
            completed: events.filter(e => e.status === "completed").length
          }
        };

        res.status(200).json(
          ApiResponse.success(stats, "Statistics retrieved successfully")
        );
      } catch (error) {
        res.status(500).json(
          ApiResponse.error("Internal server error", 500)
        );
      }
    }
  }

  export default EventController;
```

Step 3: Enhanced Routes

Edit: `src/routes/eventRoutes.js`

Replace with this:

```
// PROFESSIONAL REST ROUTES

import express from "express";
import EventController from "../controllers/eventController.js";

const router = express.Router();

// CRUD Operations
router.get("/", EventController.getAllEvents);
router.post("/", EventController.createEvent);
router.get("/stats", EventController.getStats);
router.get("/:id", EventController.getEventById);
router.put("/:id", EventController.updateEvent);
router.delete("/:id", EventController.deleteEvent);

export default router;
```

Step 4: Update Server

Edit: `server.js`

Replace with this:

```
// PROFESSIONAL REST API SERVER

import express from "express";
import eventRoutes from "./src/routes/eventRoutes.js";
import { logger, measureTime } from "./src/middleware.js";

const app = express();
const PORT = 3000;
const API_VERSION = "v1";

// ===== MIDDLEWARE =====
app.use(express.json());
app.use(logger);
app.use(measureTime);

// ===== API DOCUMENTATION =====
app.get("/", (req, res) => {
  res.json({
    name: "Event Manager API",
    version: "1.0.0",
    description: "Professional REST API for event management",
    baseUrl: `http://localhost:${PORT}/api/${API_VERSION}`,
    endpoints: {
      events: {
        list: `GET /api/${API_VERSION}/events?page=1&limit=10`,
        create: `POST /api/${API_VERSION}/events`,
        get: `GET /api/${API_VERSION}/events/:id`,
        update: `PUT /api/${API_VERSION}/events/:id`,
        delete: `DELETE /api/${API_VERSION}/events/:id`,
        stats: `GET /api/${API_VERSION}/events/stats`
      },
      filters: {
        status: "?status=upcoming",
        location: "?location=Sfax",
        search: "?search=javascript",
        minCapacity: "?minCapacity=20",
        pagination: "?page=1&limit=10"
      }
    }
  });
});

// ===== API ROUTES =====
app.use(`/api/${API_VERSION}/events`, eventRoutes);

// ===== HEALTH CHECK =====
app.get("/health", (req, res) => {
  res.json({
    status: "✅ healthy",
    timestamp: new Date().toISOString(),
    uptime: process.uptime().toFixed(2) + "s"
```

```
  });
});

// ===== ERROR HANDLERS =====

// 404 Not Found
app.use((req, res) => {
  res.status(404).json({
    success: false,
    message: `Endpoint not found: ${req.method} ${req.path}`,
    timestamp: new Date().toISOString()
  });
});

// Error handler
app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).json({
    success: false,
    message: "Internal server error",
    error: process.env.NODE_ENV === "development" ? err.message : undefined,
    timestamp: new Date().toISOString()
  });
});

// ===== START SERVER =====
app.listen(PORT, () => {
  console.log(`\n✨ Professional REST API Server`);
  console.log(` 📍 http://localhost:${PORT}`);
  console.log(` 🔗 API v${API_VERSION} at
http://localhost:${PORT}/api/${API_VERSION}`);
  console.log(`\n📋 Available Endpoints:`);
  console.log(`   GET    /api/${API_VERSION}/events`);
  console.log(`   POST   /api/${API_VERSION}/events`);
  console.log(`   GET    /api/${API_VERSION}/events/:id`);
  console.log(`   PUT    /api/${API_VERSION}/events/:id`);
  console.log(`   DELETE /api/${API_VERSION}/events/:id`);
  console.log(`   GET    /api/${API_VERSION}/events/stats`);
  console.log(`\n⏹  Press Ctrl+C to stop\n`);
});
```

☑ CHECKPOINT 2: Complete CRUD

- ☐ Event model supports filtering
- ☐ Validation service works
- ☐ ApiResponse wrapper implemented
- ☐ All HTTP status codes correct
- ☐ Error handling comprehensive
- ☐ Pagination implemented
- ☐ Controller handles all cases

🎯 TASK 3: API DOCUMENTATION WITH SWAGGER

Step 1: Install Swagger

DO THIS:

```
npm install swagger-ui-express swagger-jsdoc
```

Step 2: Create Swagger Documentation

Create file: `src/swagger.js`

Copy this:

```
// SWAGGER/OPENAPI DOCUMENTATION

import swaggerJsdoc from "swagger-jsdoc";

const options = {
  definition: {
    openapi: "3.0.0",
    info: {
      title: "Event Manager API",
      version: "1.0.0",
      description: "Professional REST API for managing events",
      contact: {
        name: "Mohamed Amine Marzouk",
        email: "med.amine.mzk@horizon-tech.tn"
      }
    },
    servers: [
      {
        url: "http://localhost:3000",
        description: "Development server"
      }
    ]
  },
  apis: ["./src/routes/eventRoutes.js"]
};

export const swaggerSpec = swaggerJsdoc(options);
```

Step 3: Add Swagger Routes

Edit: `server.js`

Add after imports:

```
import swaggerUi from "swagger-ui-express";
import { swaggerSpec } from "./src/swagger.js";
```

Add after middleware:

```
// Swagger documentation
app.use("/api-docs", swaggerUi.serve);
app.get("/api-docs", swaggerUi.setup(swaggerSpec));
```

Step 4: Update Routes with Documentation

Edit: `src/routes/eventRoutes.js`

Add comments:

```
/**
 * @openapi
 * /api/v1/events:
 *   get:
 *     summary: Get all events
 *     parameters:
 *       - in: query
 *         name: page
 *         schema:
 *           type: integer
 *       - in: query
 *         name: limit
 *         schema:
 *           type: integer
 *       - in: query
 *         name: status
 *         schema:
 *           type: string
 *     responses:
 *       200:
 *         description: List of events
 *   post:
 *     summary: Create new event
 *     requestBody:
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             type: object
 *             properties:
```

```
 *                title:
 *                  type: string
 *                date:
 *                  type: string
 *                location:
 *                  type: string
 *                capacity:
 *                  type: integer
 *      responses:
 *        201:
 *          description: Event created successfully
 */
```

☑ CHECKPOINT 3: Documentation

- ☐ Swagger UI installed
- ☐ Swagger documentation created
- ☐ API docs accessible at /api-docs
- ☐ Endpoints documented

🎯 TASK 4: COMPREHENSIVE TESTING

Step 1: Create Test Script

Create file: `test-rest-api.js`

Copy this:

```javascript
// COMPREHENSIVE REST API TESTS

const BASE_URL = "http://localhost:3000/api/v1/events";

async function request(method, path, body = null) {
  const options = {
    method,
    headers: { "Content-Type": "application/json" }
  };

  if (body) options.body = JSON.stringify(body);

  const response = await fetch(`${BASE_URL}${path}`, options);
  const data = await response.json();

  return { status: response.status, data };
}

async function test(name, fn) {
  try {
```

```
      console.log(`\n✅ ${name}`);
      await fn();
    } catch (error) {
      console.error(`❌ ${name}: ${error.message}`);
    }
  }

  async function runTests() {
    console.log("═══════════════════════════════════════");
    console.log("   REST API COMPREHENSIVE TESTS");
    console.log("═══════════════════════════════════════");

    // Test 1: Get all events
    await test("GET /events - Get all events", async () => {
      const result = await request("GET", "");
      console.log(`   Events: ${result.data.data.length}`);
      console.log(`   Status: ${result.status}`);
    });

    // Test 2: Pagination
    await test("GET /events?page=1&limit=5 - Pagination", async () => {
      const result = await request("GET", "?page=1&limit=5");
      console.log(`   Page: ${result.data.pagination.page}`);
      console.log(`   Limit: ${result.data.pagination.limit}`);
    });

    // Test 3: Filter by status
    await test("GET /events?status=upcoming - Filter", async () => {
      const result = await request("GET", "?status=upcoming");
      console.log(`   Filtered: ${result.data.data.length} events`);
    });

    // Test 4: Create event
    let createdEventId;
    await test("POST /events - Create event", async () => {
      const result = await request("POST", "", {
        title: "Test Workshop",
        date: "2026-07-15T10:00:00Z",
        location: "Sfax",
        capacity: 25
      });
      createdEventId = result.data.data.id;
      console.log(`   Created ID: ${createdEventId}`);
      console.log(`   Status: ${result.status}`);
    });

    // Test 5: Get single event
    await test("GET /events/:id - Get single", async () => {
      const result = await request("GET", `/${createdEventId}`);
      console.log(`   Event: ${result.data.data.title}`);
      console.log(`   Status: ${result.status}`);
    });

    // Test 6: Update event
```

```
  await test("PUT /events/:id - Update", async () => {
    const result = await request("PUT", `/${createdEventId}`, {
      title: "Updated Workshop Title"
    });
    console.log(`   Updated: ${result.data.data.title}`);
    console.log(`   Status: ${result.status}`);
  });

  // Test 7: Invalid data
  await test("POST /events - Validation error", async () => {
    const result = await request("POST", "", {
      title: "A" // Too short
    });
    console.log(`   Error: ${result.data.message}`);
    console.log(`   Status: ${result.status}`);
  });

  // Test 8: Not found
  await test("GET /events/9999 - Not found", async () => {
    const result = await request("GET", "/9999");
    console.log(`   Message: ${result.data.message}`);
    console.log(`   Status: ${result.status}`);
  });

  // Test 9: Statistics
  await test("GET /events/stats - Statistics", async () => {
    const result = await request("GET", "/stats");
    console.log(`   Total events: ${result.data.data.totalEvents}`);
    console.log(`   Status: ${result.status}`);
  });

  // Test 10: Delete event
  await test("DELETE /events/:id - Delete", async () => {
    const result = await request("DELETE", `/${createdEventId}`);
    console.log(`   Deleted successfully`);
    console.log(`   Status: ${result.status}`);
  });

  console.log("\n═══════════════════════════════════════");
  console.log("   ✅ ALL TESTS COMPLETE!");
  console.log("═══════════════════════════════════════");
}

// Run tests
console.log("Waiting for server...\n");
setTimeout(runTests, 2000);
```

Step 2: Run Tests

Terminal 1:

```
npm start
```

Terminal 2:

```
node test-rest-api.js
```

☑ CHECKPOINT 4: Testing Complete

- ☐ All CRUD operations tested
- ☐ Validation tested
- ☐ Error handling tested
- ☐ Status codes verified
- ☐ Pagination tested
- ☐ Filtering tested

🎁 BONUS CHALLENGES

Bonus 1: Search Events

```
// Search by title or description
GET /api/v1/events?search=javascript
```

Bonus 2: Advanced Filtering

```
// Multiple filters
GET /api/v1/events?status=upcoming&location=Sfax&minCapacity=20
```

Bonus 3: Sorting

```
// Sort results
GET /api/v1/events?sort=date
GET /api/v1/events?sort=-capacity
```

Bonus 4: Batch Operations

```
// Get multiple events
POST /api/v1/events/batch
```

```
{
  "ids": [1, 2, 3]
}
```

Bonus 5: Event Attendees

```
POST /api/v1/events/:id/attend
DELETE /api/v1/events/:id/attend/:userId
```

🆘 TROUBLESHOOTING

404 errors

- Check endpoint path
- Verify HTTP method
- Check :id parameter

422 validation errors

- Check field values
- Verify required fields
- Read error message

Status codes not correct

- Verify res.status() call
- Check response type
- Review ApiResponse usage

📦 LAB 4 DELIVERABLES

Submit:

1. ☑ `src/models/Event.js` - Enhanced model
2. ☑ `src/controllers/eventController.js` - Full controller
3. ☑ `src/routes/eventRoutes.js` - API routes
4. ☑ `src/services/validationService.js` - Validation
5. ☑ `src/utils/apiResponse.js` - Response wrapper
6. ☑ `src/swagger.js` - API documentation
7. ☑ `server.js` - Complete server
8. ☑ `test-rest-api.js` - Test suite
9. ☑ `package.json` - Dependencies

🎓 GRADING RUBRIC

Criterion	Points
CRUD Operations	20
HTTP Status Codes	15
Validation	15
Error Handling	15
API Response Format	15
Documentation	10
Testing	10
TOTAL	**100**

🎉 CONGRATULATIONS!

You've completed **LAB 4: REST API Design**!

You now can: ☑ Design professional REST APIs
☑ Implement CRUD operations
☑ Use proper HTTP status codes
☑ Validate input data
☑ Handle errors gracefully
☑ Document APIs
☑ Test thoroughly

Ready for LAB 5: MongoDB! 🚀

Instructor: Mohamed Amine Marzouk
Institution: ISITCom - PRINCE Laboratory
Email: med.amine.mzk@horizon-tech.tn
Last Updated: January 25, 2026
Status: ☑ Ready for Students